

Trabajo Práctico Integrador — Bases de Datos para Inteligencia Artificial

Plataforma educativa con asistencia inteligente (caso de uso #9)

Maira Ferrari · Sebastián Pardo

2026

Contents

Informe técnico	1
1. Descripción del caso de uso	1
2. Relevamiento de datos necesarios	2
3. Clasificación de los datos según su tipo	2
4. Modelo conceptual	2
5. Modelo de implementación según la tecnología elegida	3
6. Decisiones de normalización, embebido, referencia o desnormalización	3
7. Justificación de la tecnología seleccionada	3
8. Implementación mínima realizada	3
9. Datos de ejemplo utilizados	4
10. Consultas representativas	4
11. Propuesta para datos semiestructurados, no estructurados y vectoriales	4
12. Propuesta de arquitectura de datos	5
13. Estrategia de seguridad, permisos y aislamiento	5
14. Consideraciones de escalabilidad y rendimiento	5
15. Conclusiones	5

Informe técnico

Carrera de Especialización en Inteligencia Artificial (UBA FIUBA) — Bases de Datos para IA. Repositorio: `plataforma-educativa-asistencia-inteligente`. Este documento consolida el trabajo de las issues #1 a #16 en las 15 secciones que pide la consigna. Cada sección enlaza al documento de detalle correspondiente (todos en `docs/`, `nosql/` y `vectorial/`) para no duplicar contenido; acá se resume lo necesario para evaluar el trabajo de punta a punta.

1. Descripción del caso de uso

El caso elegido es el #9 — **Plataforma educativa con asistencia inteligente**: una institución educativa que dicta cursos y quiere sumar un asistente de IA que ayude a cada estudiante a partir de su propia actividad académica (responder consultas sobre contenidos, recomendar material, detectar riesgo académico temprano). El trabajo no construye el asistente ni el modelo de lenguaje: diseña la **capa de datos** que lo hace posible.

Actores: estudiante, docente, tutor, coordinador, administrador y el asistente (actor no humano que opera siempre con los permisos del estudiante que lo invoca). Detalle completo de contexto, actores, procesos, necesidades de información y criterios de diseño en `analisis_caso_uso.md` (issue #1).

2. Relevamiento de datos necesarios

El relevamiento se hizo por subdominio y se consolida acá:

- **Gestión académica** (usuarios, estudiantes, docentes, cursos, períodos académicos, inscripciones): datos estructurados y operacionales — identidad, matrícula y oferta de cursos, el núcleo que referencia el resto del sistema.
- **Materiales de estudio** (materiales, fragmentos): estructurado en sus campos fijos (curso, autor, tipo, nivel de acceso) y semiestructurado en su contenido variable (**metadata JSONB**).
- **Evaluación** (actividades, evaluaciones, entregas, calificaciones, progreso académico): estructurado y operacional, con **progreso_academico** como dato analítico agregado. Detalle completo, clasificado en las 7 categorías de la consigna (estructurado / semiestructurado / no estructurado, operacional / analítico, sensible, auditoría), en [relevamiento_datos.md](#) (issue #2).
- **Asistente** (conversaciones, mensajes, fuentes citadas, eventos de auditoría, recomendaciones): semiestructurado (JSONB) y vectorial (embeddings de fragmentos y de consultas frecuentes). También detallado en [relevamiento_datos.md](#).

Datos de ejemplo para las cuatro áreas en [data/ejemplos/](#) (evaluación y asistente) y en los `db/datos/*_seed.sql` de cada subdominio (ver sección 9).

3. Clasificación de los datos según su tipo

Tipo	Ejemplos en el proyecto
Estructurado	<code>usuarios</code> , <code>estudiantes</code> , <code>docentes</code> , <code>cursos</code> , <code>inscripciones</code> , <code>actividades</code> , <code>evaluaciones</code> , <code>entregas</code> , <code>calificaciones</code>
Semiestructurado	<code>materiales.metadata</code> , <code>conversaciones_asistente.mensajes</code> , <code>eventos_asistente.detalle</code> (todos JSONB)
No estructurado	Texto libre de consultas y respuestas del asistente antes de vectorizar; contenido de materiales (video, apuntes)
Operacional	Inscripciones, entregas, calificaciones, conversaciones — todo lo que se escribe en el día a día
Analítico	<code>progreso_academico</code> (agregado recalculado periódicamente)
Sensible	Datos personales de <code>estudiantes</code> ; <code>calificaciones</code> , <code>progreso_academico</code> y <code>conversaciones_asistente</code> (desempeño y dificultades individuales); materiales <code>restringido</code>
Auditoría	<code>eventos_asistente</code> (uso del asistente, derivaciones); todo cambio de calificación queda auditado en la misma transacción
Vectorial	<code>fragmento_embeddings</code> (materiales), <code>consultas_frecuentes_embeddings</code> (FAQ del asistente)

4. Modelo conceptual

Se construyó en dos partes que después se integraron:

- **Parte A** — Gestión académica y materiales de estudio: [modelo_conceptual_gestion_materiales.md](#) (issue #3).
- **Parte B** — Evaluación y asistente: [modelo_conceptual_evaluacion_asistente.md](#) (issue #4).

- **Integración:** [integracion.md](#) (issue #5) une ambas partes, resuelve las claves foráneas que quedaban pendientes entre subdominios y decide cómo unificar el modelo de embeddings de materiales (ver sección 6).

Ambos documentos incluyen el diagrama entidad-relación completo (Mermaid, se renderiza en GitHub), los atributos por entidad, las cardinalidades y las restricciones del dominio.

5. Modelo de implementación según la tecnología elegida

Toda la solución se implementa sobre un único **PostgreSQL 16 con pgvector** (`docker-compose.yml`), combinando tres representaciones:

- **Relacional:** `db/estructura/gestion_academica_*.sql` y `db/estructura/evaluacion_0{1-6}_*.sql` — tablas, PK, FK, CHECK, índices B-tree.
- **Documental (JSONB):** `materiales.metadata` (`materiales_01_materiales.sql`); `conversaciones_asistente` y `eventos_asistente` (`evaluacion_07_conversaciones_asistente.sql`, `evaluacion_08_eventos_asistente.sql`) — ver [nosql/materiales.md](#) y [nosql/asistente.md](#).
- **Vectorial (pgvector):** `fragmento_embeddings` (`materiales, materiales_vectorial_0{2,3}_*.sql`) y `consultas_frecuentes_embeddings` (`asistente, vectorial_03_consultas_frecuentes.sql`) — ver [vectorial/materiales.md](#) y [vectorial/asistente.md](#).

El modelo físico completo (tipos de dato, índices, restricciones reales) está en esos mismos scripts SQL, ejecutables tal cual contra el Postgres del `docker-compose.yml`.

6. Decisiones de normalización, embebido, referencia o desnormalización

- **Normalización relacional:** el núcleo académico y de evaluación está en 3FN (sin dependencias transitivas); se justifica caso por caso en [modelo_conceptual_evaluacion_asistente.md](#) y en los comentarios de cada script de `db/estructura/`.
- **Embebido vs. referencia (JSONB):** mensajes y fuentes citadas se embeben dentro de la conversación (sin existencia propia, se leen siempre juntos); el material referenciado desde una fuente se guarda por **referencia** (`documento_id`), no embebido, porque tiene ciclo de vida propio. Detalle completo en [nosql/asistente.md](#), sección 3.
- **Desnormalización deliberada** — **ConsultaAsistente vs. ConversacionAsistente:** se mantienen ambas representaciones (relacional liviana + documento completo) a propósito, ver [modelo_conceptual_evaluacion_asistente.md](#), sección 5.
- **Reconciliación del modelo de embeddings de materiales** (issue #5/#16): el asistente (#13) había modelado los embeddings de forma denormalizada (texto, curso y nivel de acceso duplicados en la misma fila) y materiales (#12) de forma normalizada (`material_fragmentos` + `fragmento_embeddings`, con JOIN a `materiales`). En la integración se adoptó el **modelo normalizado**: a esta escala el JOIN no pesa, y evita que un fragmento quede citable con un nivel de acceso desactualizado si el material se vuelve restringido. Ver [integracion.md](#), sección 3.

7. Justificación de la tecnología seleccionada

Desarrollada en detalle, componente por componente y contra alternativas descartadas (MongoDB, motor de grafos, vector store dedicado, Data Warehouse separado), en [seleccion_tecnologica.md](#) (issue #16). Resumen: un único PostgreSQL multi-modelo cubre los tres patrones de acceso del caso (transaccional, documental, similitud) sin la complejidad operativa ni el riesgo de inconsistencia de separar motores, a un volumen (institución educativa, no escala masiva) que no exige esa separación todavía.

8. Implementación mínima realizada

Se implementó y se **probó de punta a punta** contra el Postgres+pgvector del `docker-compose.yml` (esquema completo, reseteando el volumen y corriendo desde cero):

1. `db/estructura/*.sql` — tablas de los cuatro subdominios (16 scripts).

2. `db/datos/*.sql` — seeds de los cuatro subdominios (6 scripts).
3. `db/indices_vistas/*.sql` — vistas (ranking de estudiantes, ocupación de cursos).
4. `db/integracion/*.sql` — FKs cross-subdominio (#5) y RLS de evaluación/asistente (#17).

Orden de aplicación y verificación completa en [integracion.md](#), sección 4. Incluye roles de base de datos y Row-Level Security (`db/estructura/seguridad/*.sql`, `db/integracion/integracion_02_rls_evaluacion.sql`), verificados con `SET ROLE app_estudiante`.

9. Datos de ejemplo utilizados

- `data/ejemplos/`: 9 archivos JSON (subdominio evaluación/asistente), 3-5 registros por entidad, con IDs legibles y consistentes entre sí (`est-*`, `doc-*`, `cur-*`, `mat-*`, `act-*`, `ent-*`, `cal-*`, `conv-*`).
- `db/datos/gestion_academica_seed.sql`: 6 estudiantes, 3 docentes, 2 cursos, 10 inscripciones.
- `db/datos/materiales_seed.sql` y `materiales_vectorial_seed.sql`: 6 materiales y sus fragmentos/embeddings.
- `db/datos/evaluacion_seed.sql`, `nosql_asistente_seed.sql`, `vectorial_asistente_seed.sql`: equivalentes SQL de `data/ejemplos/`, más el caché de FAQ.

Los mismos IDs se usan en todos los subdominios para que las relaciones cierren en la integración (ver `analisis_caso_uso.md`, sección 6).

10. Consultas representativas

24 consultas SQL en total (mínimo pedido: 5), todas probadas contra los datos de ejemplo:

Archivo (en <code>db/consultas/</code>)	Subdominio	#	Cubre
<code>gestion_academica_consultas.sql</code>	Gestión académica (#8)	5	JOIN, N:M, agregación, vista + decisión, <code>RANK()</code>
<code>materiales_consultas.sql</code>	Materiales, JSONB (#10)	5	extracción, contención <code>@></code> , <code>jsonb_array_elements</code> , seguridad, agregación
<code>materiales_vectorial_consultas.sql</code>	Materiales, vectorial (#12)	3	RAG con filtro de acceso, auditoría, trazabilidad de fuente
<code>evaluacion_consultas.sql</code>	Evaluación (#9)	5	filtrado, JOIN, agregación + <code>HAVING</code> , decisión, vista + <code>RANK()</code>
<code>nosql_asistente_consultas.sql</code>	Asistente, JSONB (#11)	5	documento completo, <code>jsonb_array_elements</code> , contención <code>@></code> , agregación, filtro por fecha
<code>vectorial_asistente_consultas.sql</code>	Asistente, vectorial (#13)	3+	RAG, caché de FAQ, auditoría; incluye <code>EXPLAIN ANALYZE</code>

11. Propuesta para datos semiestructurados, no estructurados y vectoriales

- **Semiestructurados (JSONB)**: contenido de materiales (`materiales.metadata`, variable según tipo), conversaciones del asistente (mensajes y fuentes embebidos) y eventos de auditoría (`detalle` variable según `tipo_evento`). Justificación completa en [nosql/materiales.md](#) y [nosql/asistente.md](#).
- **No estructurados**: el texto de materiales en video/PDF y el texto libre de consultas y respuestas antes de vectorizar — no se persisten como tales, se procesan para generar fragmentos y embeddings.

- **Vectoriales:** fragmentos de materiales (para RAG) y consultas frecuentes (caché semántico de FAQ). Qué se vectoriza, qué metadatos se guardan, cómo se vincula con el dato original, qué consultas de similitud se esperan y qué riesgos de acceso indebido existen, desarrollado en [vectorial/materiales.md](#) y [vectorial/asistente.md](#), con un ejemplo real de EXPLAIN ANALYZE en este último.

12. Propuesta de arquitectura de datos

Arquitectura en capas sobre el único PostgreSQL multi-modelo (no se justifica Data Lake ni Lakehouse al volumen esperado): ingesta vía la API de la plataforma, almacenamiento operacional (relacional + documental + vectorial), almacenamiento analítico (`progreso_academico`, agregado por batch) y consumidores (asistente RAG, dashboard docente, alertas de riesgo). Incluye manejo de concurrencia (transacciones calificación+auditoría, `UNIQUE` para reintentos de entrega, `UPSERT` para el recálculo de progreso). Diagrama y detalle completos en [arquitectura_evaluacion_asistente.md](#) (issue #15); la integración de ambos subdominios en un único diagrama de arquitectura general queda como mejora pendiente (ver sección 15).

13. Estrategia de seguridad, permisos y aislamiento

Roles de base de datos `NOLOGIN` de menor privilegio (`app_estudiante`, `app_docente`, `app_coordinador`, `app_administrador`) y Row-Level Security con `app_estudiante_id` (deny-by-default: sin identidad seteada no se ve ninguna fila). Cubre gestión académica y materiales (`db/estructura/seguridad_*.sql`, issue #14) y, desde la revisión cruzada, también evaluación y asistente (`db/integracion/integracion_02_rls_evaluacion.sql`, issue #17): entregas, calificaciones, progreso académico y conversaciones quedan acotadas al propio estudiante, verificado con `SET ROLE app_estudiante`. La búsqueda vectorial nunca es la única barrera: siempre se combina con el filtro relacional de `materiales.nivel_acceso`. Detalle completo en [seguridad.md](#).

Limitación conocida: no existe todavía una política RLS por **docente** (que un docente solo vea los cursos que dicta); hoy accede por `GRANT` amplio sin acotar por `docente_titular_id`. Ver sección 15.

14. Consideraciones de escalabilidad y rendimiento

Qué crece más (`entregas`, `conversaciones_asistente`, `eventos_asistente`, con el uso diario), qué consultas son críticas (búsqueda vectorial RAG, ranking de progreso), qué índices ya están creados (B-tree, GIN, HNSW), qué podría particionarse (`conversaciones/eventos` por fecha), qué ya está precalculado (`progreso_academico`) y qué componentes podrían separarse a mayor escala (servicio de embeddings, si el volumen de fragmentos creciera a millones). Desarrollado en [arquitectura_evaluacion_asistente.md](#), sección 3, y en [vectorial/asistente.md](#), sección 5 (elección de HNSW sobre IVFFlat, con un EXPLAIN ANALYZE real que muestra por qué el planner no usa el índice vectorial a este volumen de datos de ejemplo, y cuándo empezaría a usarlo).

15. Conclusiones

El caso de uso #9 no necesitaba, a su volumen actual, más que un único motor de base de datos bien diseñado: la decisión de fondo del trabajo fue **elegir el modelo de datos por la forma de cada dato** (relacional para lo transaccional, documental para lo variable, vectorial para la similitud) en vez de forzar todo a una sola representación o, en el otro extremo, sumar motores especializados sin necesidad. El diseño resultante es coherente entre los cuatro subdominios (mismas convenciones de nombres e IDs, mismo patrón de seguridad) e íntegramente verificado contra una base real, no solo documentado en diagramas.

Limitaciones y mejoras pendientes (quedan explícitas, no ocultas):

- Aislamiento RLS por **docente** (solo sus cursos) no implementado, ni en materiales ni en evaluación — hoy los roles docentes acceden sin ese filtro fila a fila.
- El diagrama de arquitectura general que une ambos subdominios en una sola imagen no se consolidó (quedan dos diagramas Mermaid, uno por subdominio).

- Los embeddings usados en toda la implementación son **sintéticos** (vectores de 8 dimensiones agrupados por tema a mano, no generados por un modelo real) — suficiente para validar el diseño y las consultas, no para medir calidad de recuperación semántica real.
- El recálculo de **progreso_academico** se describe como job periódico pero no se implementó el job en sí (está fuera del alcance de la capa de datos, según el enunciado).